Audible Lava / DSO v4.11 — RAVE timbre transfer: implementation companion

Sam Leventhal * November 2026 * companion to blob_synth_rave.scd, rave_layer.py, test_rave_layer.py and the v4.11 controller.py / audible_lava.py

1. What this release does

v4.11 adds a final stage after the fully formed lava mix: the output of every voice, hit, melody note, strike and pad is passed through one or more pre-trained RAVE models — RAVE being the Realtime Audio Variational autoEncoder (Caillon & Esling, IRCAM) — running inside SuperCollider through the nn.ar extension (nn_tilde's SuperCollider port). Each model re-synthesises the lamp as if heard through its own training set: a dawn chorus, an organ archive, humpback whales, a VCTK voice. The layer is **off by default**: the lamp plays the v4.10 rich SynthDefs untouched until the player turns it on. Nothing in tracking, routing, the score governor, the pattern language, the melody engine or synth_palette is modified; the layer is two extra inserts and a third tab in the keyboard window.

2. Signal path

Node order in group 0 is execution order. Python spawns every voice, hit and pad at the head; the v4.10 master space sits at the tail (node 900). The RAVE nodes are added with *addBefore* 900 (or at the tail when --no-space), so the reverb and Greyhole treat the transferred timbre like any other voice.

```
[voices, hits, pads ... at the HEAD]          -> bus 0 (dry mix)
\raveSplit        node 910   In.ar(0) -> Out.ar(64)            copy for the models
                             ReplaceOut.ar(0, dry * (1 - mix))  scale the dry mix (lagged 0.5 s)
\raveXfer_<key>   920 + i    In.ar(64) -> L+R sum * inGain -> NN(key, \forward) -> Pan2 -> Out.ar(0)
                             one node per ACTIVE model; amp = mix / sqrt(n_active); gate -> fade -> free
\lavaMaster       node 900   the space hears dry + wet
```

Several models can be on at once and run **in parallel**, each reading the same dry copy and adding its own wet signal — birds + organ is the lamp heard through two ears, not birds-of-an-organ. A series chain was rejected: RAVE output fed into a second RAVE compounds artefacts and latency. The split is spawned once at startup and stays (dry = 1 is a transparent passthrough); model nodes exist only while their model is on, because in nn.ar every UGen holds its own instance of its model and twelve instances at once would be twelve inference threads.

3. Why one SynthDef per model

nn.ar loads a model's *description* once (NN.load, on scsynth, index cached in sclang) and the UGen references model and method by integer index at SynthDef build time. A single SynthDef with a selector would therefore have to instantiate every model it could select. blob_synth_rave.scd instead loads each description it finds, builds \raveXfer_<key> for each one that loaded, and writes rave_status.json next to the models (loaded / missing / failed per key, trained sample rate, server rate). Python reads that file to annotate the tab and never blocks on it: an /s_new for a SynthDef that does not exist is one line in the post window. Every reference to the extension goes through \NN.asClass, so the file compiles without nn.ar installed (the v4.10 convention for sc3-plugins).

4. The models

Key (home-row key)	File	Buffer / rate / latent	Note
birds (a)	birds_dawnchorus_b2048_r48000_z8.ts	2048 / 48 k / 8	
crozzoli (s)	crozzoli_bigensemblesmusic_18d.ts	? / ? / ?	no b/r/z in the name; nn.ar reports its minBufferSize and rate on load
guitar (d)	guitar_iil_b2048_r48000_z16.ts	2048 / 48 k / 16	
humpbacks (f)	humpbacks_pondbrain_b2048_r48000_z20.ts	2048 / 48 k / 20	

Key (home-row key)	File	Buffer / rate / latent	Note
magnets (g)	magnets_b2048_r48000_z8.ts	2048 / 48 k / 8	
marine (h)	marinemammals_pondbrain_b2048_r48000_z20.ts	2048 / 48 k / 20	
mrp (j)	mrp_strengjavera_b2048_r44100_z16.ts	2048 / 44.1 k / 16	trained at 44.1 k; server runs at 48 k (see §7)
organ (k)	organ_archive_b2048_r48000_z16.ts	2048 / 48 k / 16	
voicemulti (l)	voice-multi-b2048-r48000-z11.ts	2048 / 48 k / 11	
vctk (;)	voice_vctk_b2048_r44100_z22.ts	2048 / 44.1 k / 22	trained at 44.1 k (see §7)
vocalset (')	voice_vocalset_b2048_r48000_z16.ts	2048 / 48 k / 16	
water (\)	water_pondbrain_b2048_r48000_z16.ts	2048 / 48 k / 16	

Files live in /Users/multivax/Documents/RAVE_PreTrained_Models (rave_layer.RAVE_MODEL_DIR; the environment variable AUDIBLE_LAVA_RAVE_DIR overrides it, and ~raveModelDir on the SuperCollider side). The key list is declared once in rave_layer.MODELS and mirrored in ~raveModels; test_rave_layer.py fails if they drift.

5. Controls

Where	Key	Action
keyboard window	TAB	cycles PERFORMANCE → EDIT → RAVE → PERFORMANCE (the RAVE tab exists only when the layer is installed)
RAVE tab	a s d f g h j k l ; ' \	toggle model 0..11 on/off (selection is kept while the layer is off)
RAVE tab	space	whole layer on/off (model nodes are spawned / gated)
RAVE tab	up / down, = / -	wet mix ±0.05, ±0.25 (dry × (1 – mix), each model at mix / √n)
RAVE tab	left / right	input gain into the models ÷ / × 1.41 (RAVE wants a healthy level; the lamp mix is often quiet)
RAVE tab	z	all models off
RAVE tab	r	re-read rave_status.json and re-send the split (after evaluating the .scd late)
RAVE tab	1 … 0	slot select still works, as in EDIT
video window	a	layer on/off (same as space in the tab); HUD flag [A], second status line 'RAVE on mix=… [keys]'
command line	--no-rave / --rave-on	leave the layer out entirely / start with it on (models still chosen from the tab)

The tab lists every model with its key, an [x]/[ ] box, its trained rate (a '!' marks a mismatch with the server rate), and 'file missing' / 'failed to load' / '(not verified)' from rave_status.json; a warning line says when the status file is absent (evaluate the .scd) or nn.ar is not installed.

6. Wiring and operation

File	Status	Role
blob_synth_rave.scd	NEW	NN.load of the twelve descriptions, one \raveXfer_ SynthDef per loaded model, \raveSplit, rave_status.json, ~raveOn/~raveOff helpers and manual tests
rave_layer.py	NEW	RaveModel table, RaveStatus reader, RaveLayer (split, model nodes, levels, controller events, panel, HUD, close), install()
controller.py	MODIFIED	RAVE_* event types, RAVE_MODE, enable_rave_tab(), three-way TAB cycle, RAVE tab keys and rendering; everything else unchanged
audible_lava.py	MODIFIED	imports rave_layer; install() after synth_palette.install (master node id); dispatch of RAVE_* events; rave.update() per frame; 'a' key; HUD; close/free_split at exit; flags
test_rave_layer.py	NEW	14 tests, no SuperCollider/camera/pygame (Results: 14 passed)

File	Status	Role
build_companion_v411.py	NEW	this document

SuperCollider load order: blob_synth.scd → blob_synth_hits.scd → melody_hit.scd → blob_synth_rich.scd → **blob_synth_rave.scd**, then python audible_lava.py. First run on macOS, because the nn.ar release binaries are unsigned: *NN.deQuarantine; s.reboot;*. Expect in the post window one line per model ('+ organ : loaded (rate 48000 Hz, minBufferSize 2048)'), then 'Audible Lava v4.11: RAVE layer ready -- N of 12 models loaded'. Node ids: split 910, models 920–931 (master 900, pre-sound 901–903, VoiceManager from 1000). Private bus 64/65.

```
# minimal integration (what the v4.11 audible_lava.py does)
rave = rave_layer.install(voices, master_node=rig.master.node_id if rig.master else None,
                          enabled=False, mix=0.7, in_gain=1.0)
keyboard.enable_rave_tab(rave.panel_lines, rave_layer.RAVE_MODEL_KEYS)
...   # controller events whose type starts with "RAVE_":
rave.handle_controller_event(ev)
...   # per frame (cheap: re-reads rave_status.json when SC rewrites it):
rave.update()
...   # at exit: gate the models, let the fades pass through the master, then free the split
rave.close(); time.sleep(3.5); rig.close(); rave.free_split()
```

7. Risks and tradeoffs

Risk	Assessment / mitigation
Latency	Buffering 2048/48 k = 43 ms, inference 15–30 ms per buffer on a 2016 i5 (nn.ar README), plus RAVE's cached-convolution delay of ~650 ms for non-causal v2 models. The wet layer trails the dry by about 0.7 s. On 90–180 s blob cycles this reads as a halo; at mix = 1 the dry is gone and the offset is invisible. No alignment delay is applied to the dry path — it would be a per-model guess, and DelayN with a changing time clicks.
Sample rate	scsynth runs at 48 kHz; mrp and vctk were trained at 44.1 kHz. nn.ar does not resample, so those two run ~8.8 % off-rate: the model hears the lamp a semitone and a half low and its output plays back the same amount high, which cancels in pitch but tilts the spectrum. Flagged with '!' in the tab. A proper fix is a resampling pair around the UGen or re-exporting the models at 48 k.
CPU	One RAVE v2 model at 2048 is 15–30 ms of inference per 43 ms buffer on an old i5; Apple silicon handles 2–4 comfortably. The Raspberry Pi 5 is marginal and needs nn.ar built from source against an arm64 libtorch (no release build). Keep models off when not performing; the split alone is negligible.
Node destruction blocks DSP	nn.ar waits for the inference thread when a UGen is freed, which can glitch the DSP chain briefly. Model nodes are therefore gated (1 s fade, then self-free) rather than freed, so the glitch, if any, lands in silence. Startup is the mirror: a new node is silent while its thread loads the model; the 1.5 s fade-in covers it.
Level	RAVE output can be hot and models differ in gain. Each node has a Limiter at 0.9; the bank is levelled at mix / √n; the master limiter follows. inGain exists because the lamp's mix is often well below the level the models were trained on — raise it until the model 'speaks', then back off.
Status file	If rave_status.json is absent or stale, Python still sends /s_new (the only cost is a post-window line); the tab says '(not verified)'. Press r after evaluating the .scd late. The file is rewritten on every evaluation; a 2 s mtime poll picks it up.
/g_freeAll at startup	As with the master, Python spawns the split after VoiceManager; ~raveOn is for language-only sessions and places nodes at the tail (after a language-spawned master, which is fine for listening).
Crossfade discipline	No handover logic was added. Dry gain and model amplitudes are lagged kr controls of a static setting chosen by the player; model on/off is a gate on an ASR envelope. The RAVE layer is an instrument choice, not a governed quantity.

8. Tests

test_rave_layer.py covers: the model table equals the twelve shipped files with unique keys and twelve home-row keys; file-name parsing (both separators; the unlabelled crozzoli model); the .scd mirrors the Python list, bus number and asClass convention; level and placement helpers; install spawns the split before the master (or at the tail) as a passthrough; models only spawn nodes while the layer is on and the selection survives turning it off; bank levelling at mix / √n and gating of a single model; mix and input-gain clamps; every RAVE_* controller event; status-file states,

44.1 k flagging, mtime polling and the nn.ar-missing warning; close-then-free ordering; the TAB cycle only includes RAVE once enabled; every RAVE-tab key maps to the right event with slots still selectable and the home row still a piano in PERFORMANCE; and the panel renders without a display. Run: **python test_rave_layer.py**.

9. Where this could go

The layer is deliberately a player's instrument choice. Two obvious governed extensions, both small: (a) map a ScoreState quantity onto *mix* (phrase_position, so a phrase dissolves into its transferred timbre toward the boundary, or ambiguity) through a lagged control, exactly as the master space is driven; (b) use \encode / \decode instead of \forward and let blob observables perturb the latent vector — nn.ar exposes both methods, and latent dimension z8–z22 per model is in the file names. Both are additions to rave_layer.update(); the SynthDef contract would gain a `latentMod` control.